

Remonter les signaux vers Elastic

Une architecture pragmatique pour les applications et
l'infrastructure

Applications Java, Kubernetes, systèmes, Kafka, MongoDB et PostgreSQL

Plan de la revue

01 · Comprendre

Objectif et périmètre

Sources observées, signaux attendus et critères de réussite.

02 · Acheminer

Architecture et chemins

EDOT, Collecteur Edge, Kafka, Backend et Elasticsearch.

03 · Observer

Logs, métriques et traces

Ce que chaque famille de signaux apporte au diagnostic.

04 · Exploiter

Déploiement et contrôle

IaC, topologie, responsabilités et vérifications reproductibles.

05 · Prouver

Kibana et APM

Dashboards OTel, service Java, inventaire et corrélation.

06 · Décider

Bilan et évolutions

Compromis validés, limites connues et prochaines étapes.

Décision attendue

L'architecture proposée couvre les sources du POC en combinant OpenTelemetry / EDOT et Elastic Agent, avec un chemin d'exploitation vérifiable dans Elastic.

Couverture

Logs, métriques, traces et APM pour les applications Java et l'infrastructure.

Simplicité

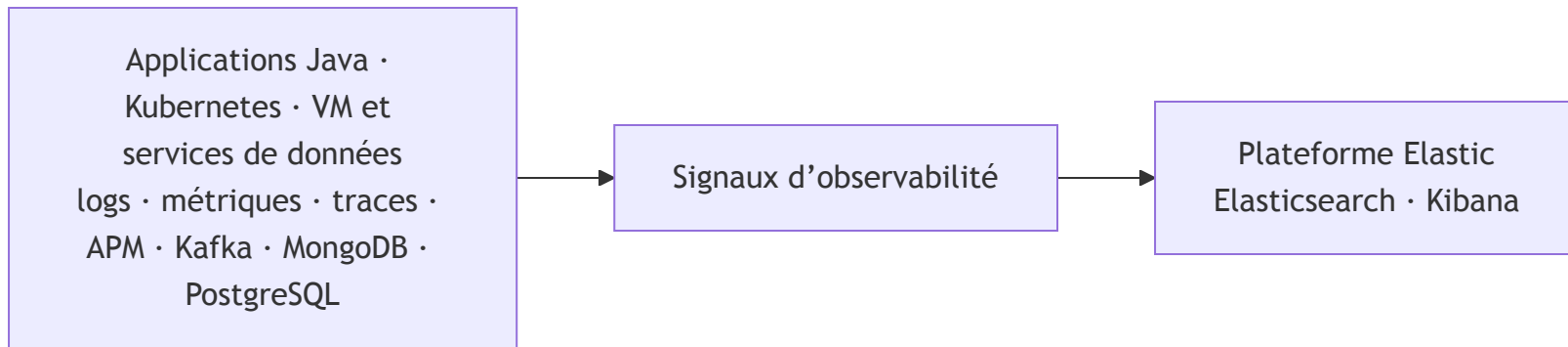
Un collecteur adapté à chaque famille de sources, avec des responsabilités lisibles.

Vérifiabilité

Chaque flux possède un transport, une destination et un contrôle observable.

La question est de valider le compromis, pas de choisir un collecteur unique par principe.

Périmètre observé

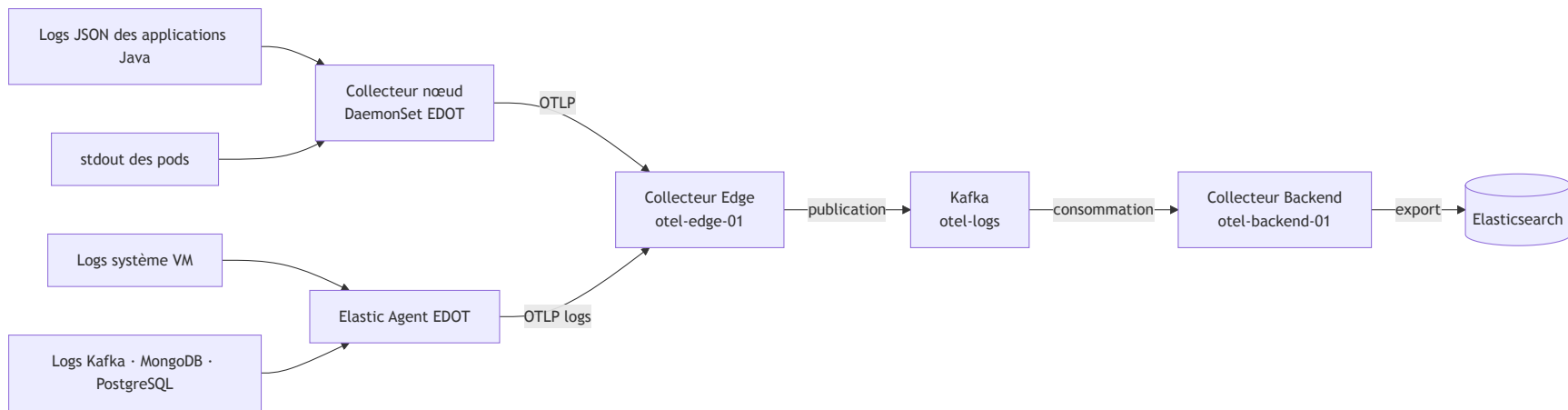


Applications et Kubernetes : rester proche d'OpenTelemetry, conserver la corrélation et utiliser Kafka comme tampon des signaux du cluster.

VM et intégrations : réduire la configuration spécifique en utilisant Elastic Agent EDOT standalone, configuré par Ansible et exportant en OTLP vers le Collecteur Edge.

1. Logs : rendre les événements consultables

Le premier besoin est de collecter les événements sans perdre leur contexte.



- Les logs applicatifs restent corrélables avec `trace.id` et `span.id`.
- Les logs Kubernetes sont enrichis avec le contexte du pod et du namespace.
- Les logs des VM et des services de données sont collectés par Elastic Agent EDOT, installé et configuré par Ansible.

Preuve attendue : événements visibles dans Kibana avec source, hôte, namespace et contexte de corrélation.

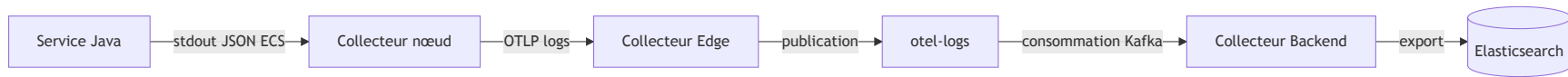
Logs applicatifs et Kubernetes

Collecte

- Les services Java écrivent des logs JSON au format ECS.
- Le Collecteur nœud (DaemonSet) lit les sorties des pods.
- `k8sattributes` ajoute le contexte Kubernetes.

Chemin

- EDOT envoie les événements en OTLP.
- Le Collecteur Edge les route vers `otel-logs`.
- Le Collecteur Backend consomme Kafka et exporte vers Elasticsearch.

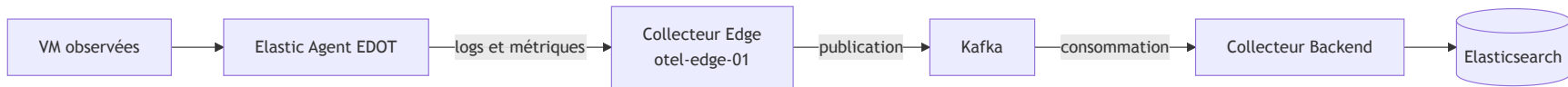


Le format et les identifiants de corrélation sont préparés avant le buffer.

Logs d'infrastructure et des services de données

L'Elastic Agent complète la couverture lorsque la source est une VM ou une intégration Elastic existante.

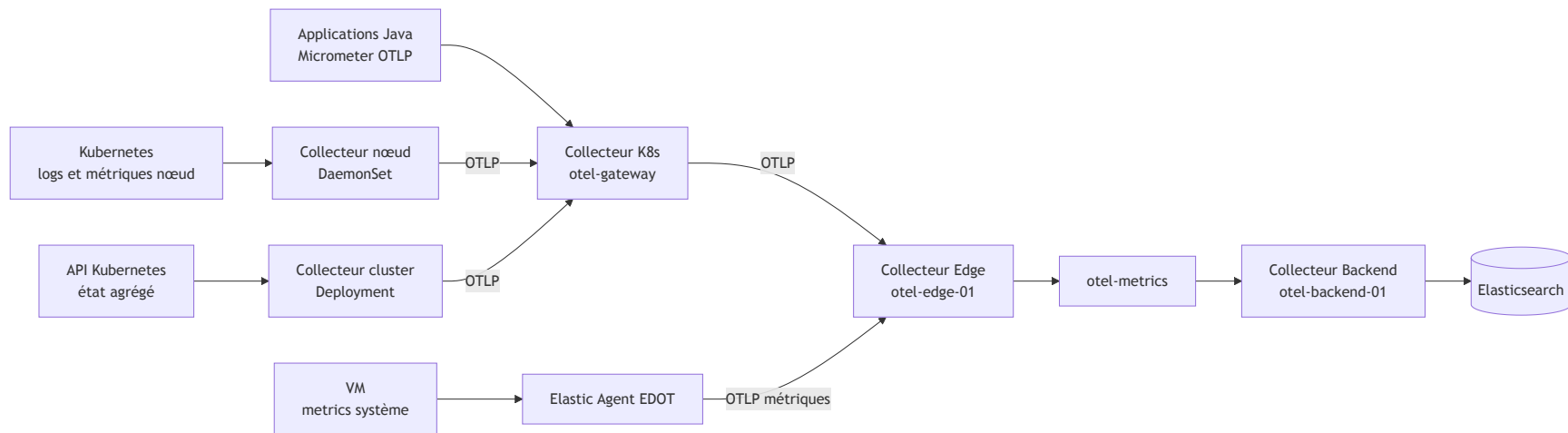
Source	Collecte	Destination
Système des VM	Elastic Agent EDOT standalone	Collecteur Edge, Kafka, Backend, Elasticsearch
Kafka	Elastic Agent EDOT standalone	Collecteur Edge, Kafka, Backend, Elasticsearch
MongoDB	Elastic Agent EDOT standalone	Collecteur Edge, Kafka, Backend, Elasticsearch
PostgreSQL	Elastic Agent EDOT standalone	Collecteur Edge, Kafka, Backend, Elasticsearch



L'Elastic Agent des VM envoie ses logs et métriques au Collecteur Edge. Ces données suivent ensuite Kafka et le Collecteur Backend avant leur indexation dans Elasticsearch.

2. Métriques : mesurer l'état et la capacité

Les métriques combinent les mesures applicatives, Kubernetes et système.



- Les métriques applicatives sont exportées en OTLP par Micrometer.
- Les métriques Kubernetes et hôte sont collectées par le Collecteur nœud ; le Collecteur cluster publie l'état agrégé via l'API Kubernetes.
- Les métriques VM et services de données sont collectées par Elastic Agent EDOT selon la configuration déclarée pour les VM.

Preuve attendue : séries applicatives et infrastructure consultables avec leurs dimensions utiles.

Métriques applicatives et Kubernetes

Applications Java

Les services `order-service`, `inventory-service` et `restock-service` exportent leurs métriques Micrometer en OTLP vers `otel-gateway`.

Cluster Kubernetes

Le Collecteur nœud et le Collecteur cluster collectent les métriques des pods, des nœuds et de l'état agrégé, puis les transmettent à `otel-gateway`.

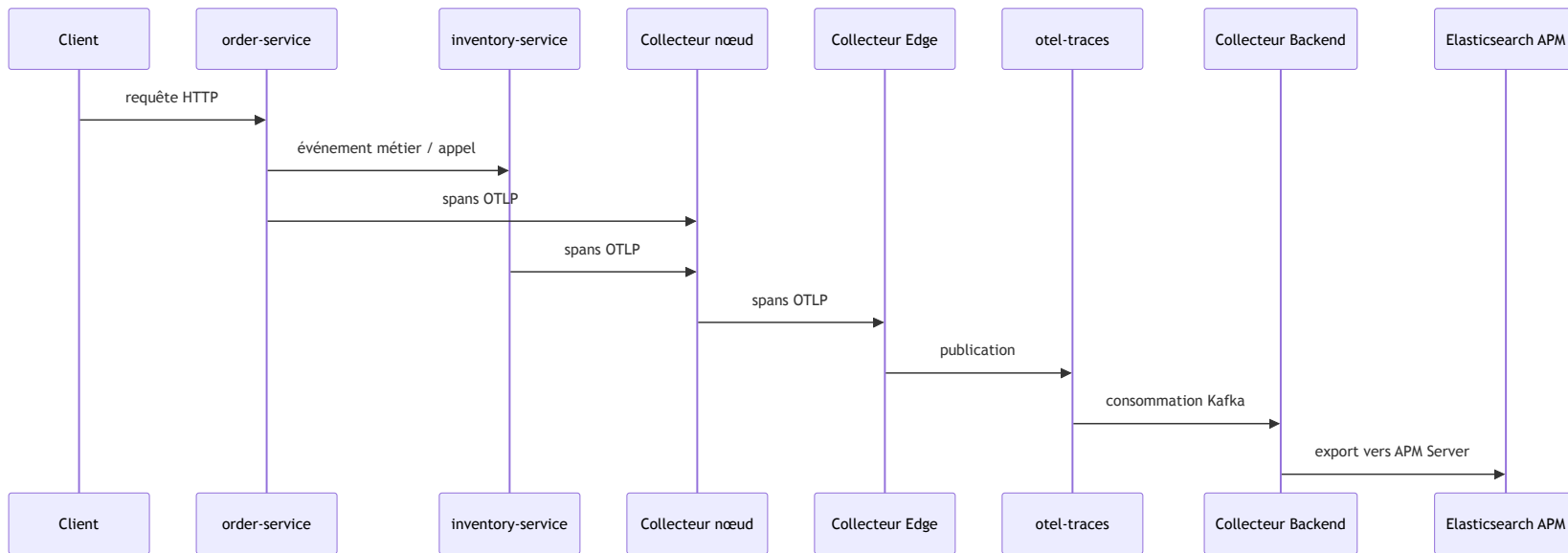
Les métriques APM produites par l'agent Java ne sont pas exportées en double : Micrometer OTLP reste le point de référence pour les métriques applicatives.

Preuve Kibana : Kubernetes OTel

Le dashboard expose l'état du cluster, la santé des nœuds et les séries de CPU, mémoire et disque produites par la collecte OTel.

3. Traces et APM : suivre une transaction

Les traces Java restent sur le chemin OpenTelemetry / OTLP, avec un buffer Kafka séparé et une exportation vers les données APM Elastic.



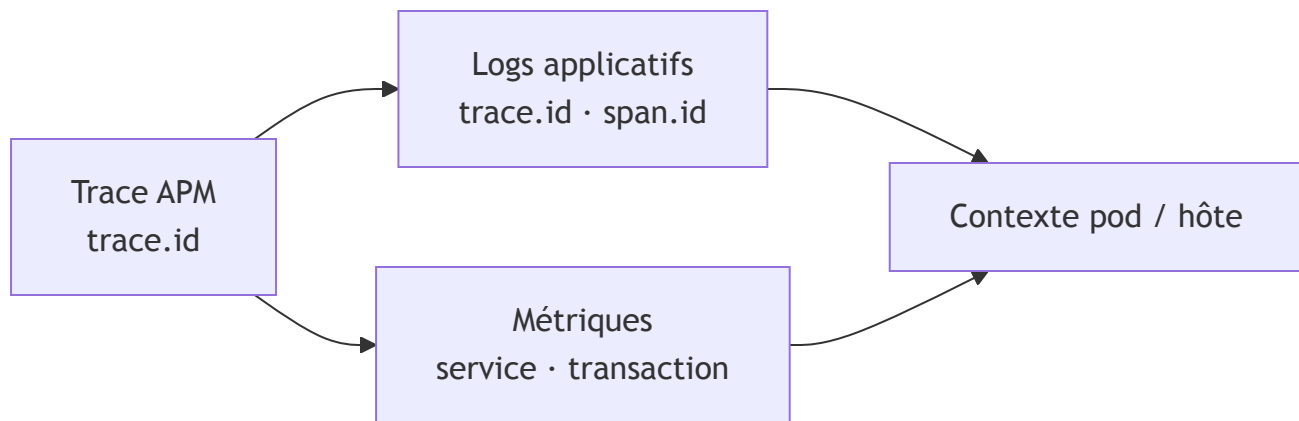
- L'agent Java est injecté dans les services et conserve les noms de service et d'environnement.
- Les événements Kafka rendent visible le fan-out entre commande, stock et réassort.
- Les identifiants de trace permettent de retrouver les logs associés.

Preuve Kibana : APM des services Java

La vue APM confirme la présence des trois services applicatifs et fournit leur débit, leur latence et leur taux d'échec pour poursuivre le diagnostic.

Corrélation : passer du signal au diagnostic

La valeur opérationnelle vient de la navigation entre signaux, pas de leur stockage séparé.



Identifier la transaction lente

Retrouver les événements
associés

Relier l'impact à
l'infrastructure

La recette Kibana devra vérifier ces trois navigations sur un même scénario métier.

Pourquoi deux chemins de collecte ?

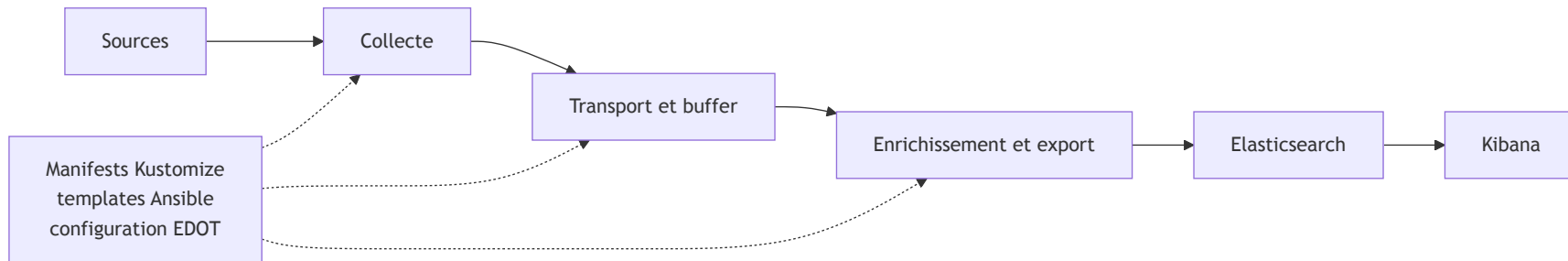
Les chemins sont séparés par responsabilité, pas par signal.

Famille de sources	Collecteur	Transport	Usage
Applications Java	Agent Java, Micrometer et Gateway OTLP	OTLP puis Kafka	Traces, logs et métriques applicatives
Kubernetes	Collecteur nœud, Collecteur cluster et Gateway	OTLP puis Kafka	Logs, pods, nœuds et métriques de cluster
VM et services de données	Elastic Agent EDOT standalone	OTLP vers Edge puis Kafka	Système, Kafka, MongoDB et PostgreSQL

Fil directeur : OpenTelemetry / OTLP pour les signaux produits par les applications et le

Simplification : Elastic Agent EDOT standalone pour les VM et les services de

Une chaîne déclarative et contrôlable

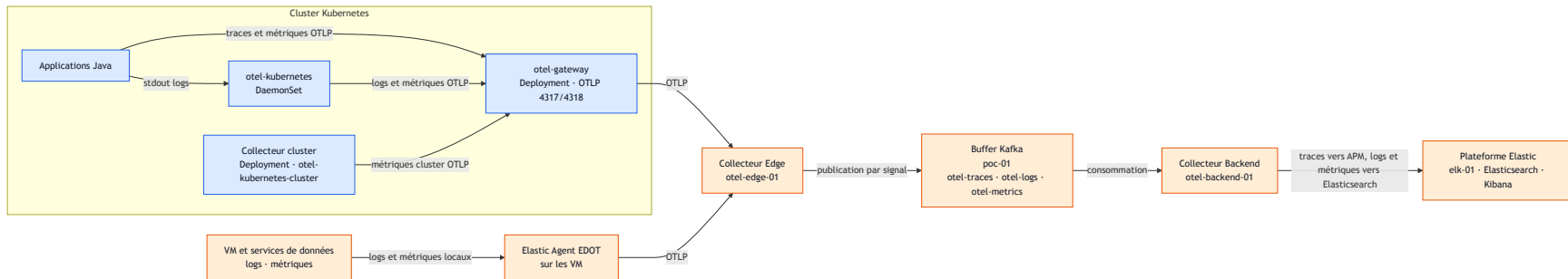


Contrôles disponibles depuis la racine :

```
make architecture-status
make kubernetes-validate
make ansible-validate
make otel-validation
make dashboards-verify
```

Une évolution durable doit être déclarée dans le dépôt puis vérifiée sur le chemin observable.

Topologie complète



Kafka tamponne les signaux. Le plan de données passe par Edge, Kafka et Backend ; les collecteurs sont configurés par laC.

Déploiements vérifiés

Les manifests et rôles de provisionnement confirment la répartition suivante.

Périmètre	Déploiement vérifié	Fonction
Kubernetes	<code>otel-kubernetes</code> DaemonSet	Logs stdout, métriques hôte, kubelet et pods
Kubernetes	Collecteur cluster (<code>otel-kubernetes-cluster</code>)	État agrégé du cluster
Kubernetes	<code>otel-gateway</code> Deployment et Service	Réception OTLP sur <code>4317</code> et <code>4318</code>
<code>otel-edge-01</code>	Collecteur EDOT et HAProxy	Entrée OTLP et publication Kafka
<code>poc-01</code>	Kafka	Topics <code>otel-traces</code> , <code>otel-logs</code> , <code>otel-metrics</code>
<code>otel-backend-01</code>	Exporteur EDOT Kafka	Export vers APM Server / Elasticsearch
<code>elk-01</code>	Elasticsearch, Kibana et APM Server	Stockage et consultation

Les collecteurs Kubernetes sont configurés par Kustomize. Le chemin des signaux reste Gateway, Edge, Kafka, Backend et Elastic.

Preuve Kibana : agents Fleet

Fleet rend observable l'état des agents qui collectent les logs et métriques des VM et des services de données.

Recette de validation

La validation finale doit suivre un scénario métier de bout en bout.

1. Déclencher une
commande

2. Suivre la trace

3. Retrouver les logs

4. Vérifier les
métriques

Vérification

Résultat attendu

Logs Java et Kubernetes

source et contexte enrichis

Logs système et intégrations

données visibles dans les data streams Elastic

Métriques

séries applicatives et infrastructure présentes

APM

trace distribuée et services associés

Corrélation

navigation par `trace.id` et `span.id`

Preuve Kibana : dashboard System OTel

Ouvrir **OTel Host Details - Metrics** avec une fenêtre temporelle de 15 minutes : les métriques sont issues de `metrics-hostmetricsreceiver.otel-*` .

Preuve Kibana : dashboard PostgreSQL OTel

Ouvrir **PostgreSQL OTel Connections** avec une fenêtre temporelle de 15 minutes. Les données proviennent de `metrics-postgresqlreceiver.otel-*`.

Preuve Kibana : dashboard Kafka OTel

Ouvrir **Kafka OTel Overview** avec une fenêtre temporelle de 15 minutes. Les données proviennent de `metrics-kafkametricsreceiver.otel-*`.

Preuve Kibana : dashboard MongoDB OTel

Ouvrir **MongoDB OTel Overview** avec une fenêtre temporelle de 15 minutes. Les données proviennent de `metrics-mongodbreceiver.otel-*`.

Preuve Kibana : service Java dans APM

La vue détaillée de `order-service` expose directement la latence, le débit, les transactions HTTP et le taux d'échec du service Java.

Trajectoire d'évolution

L'architecture actuelle reste volontairement pragmatique.

Maintenant

Deux chemins lisibles, avec une couverture complète et des contrôles associés.

Ensuite

Suivre la roadmap Elastic et réévaluer les capacités communes de collecte, d'intégration et d'export.

Condition

Uniformiser uniquement si cela réduit réellement la charge opérationnelle sans réduire la couverture.

L'uniformisation est une piste long terme, pas une contrainte de validation de la première version.

Décisions à valider

1. Les applications Java et Kubernetes restent sur le chemin OpenTelemetry / EDOT avec Kafka.
2. Les VM, Kafka, MongoDB et PostgreSQL sont couverts par Elastic Agent EDOT standalone.
3. Les trois signaux sont séparés dans Kafka pour préserver un traitement explicite.
4. La validation se fait par la chaîne complète : source, collecte, transport, Elasticsearch et Kibana.
5. L'uniformisation future dépend des capacités apportées par la roadmap Elastic.

Objectif de la revue : valider ce périmètre et identifier les contrôles manquants avant d'aller plus loin.